Now we C++

- By Aditya Varma

Preface

Welcome to Let's Learn C++

This book is written to help you learn C++ as a skill, not as a subject.

C++ is a language that gives you both control and structure. It allows you to work close to how programs actually run in memory, while also providing powerful tools to design large, organized systems. Because of this balance, learning C++ builds a deep understanding of programming that makes other languages easier to grasp.

The chapters in this book are arranged carefully. Each concept is introduced only when it is needed, and every idea is supported by simple programs that explain why something works, not just how to write it.

C++ can feel vast at first. It has many features, and it does not force you into one way of thinking. This book simplifies that journey by guiding you step by step, starting from the basics and gradually moving toward structured and object-oriented programming.

No prior programming knowledge is assumed. You are expected only to read, practice, and stay curious. This book is meant to be read slowly, like a guide. Type the programs. Modify them. Break them. Observe what changes.

Take your time. Ask questions. Make mistakes. That is how real understanding is built.

-Aditya Varma

License & Credits

Credits:

Certain icons, illustrations, and portions of reference content are adapted from publicly available educational platforms, including the Coding X mobile application. Proper credit is given where applicable. All original rights remain with their respective creators, and this material is used solely for educational purposes.

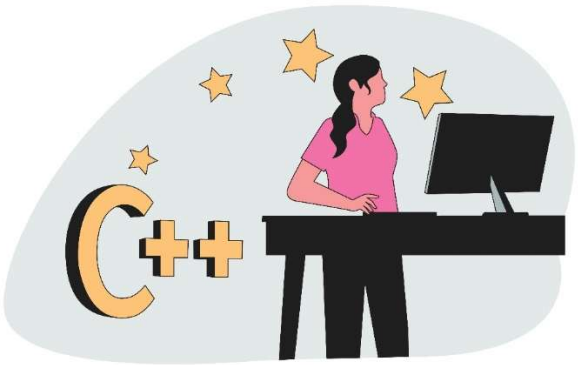
Index

Getting Started with C++

C++ is a general-purpose programming language that sits at an interesting intersection: it gives you the power to work close to the machine while also letting you build large, structured, and maintainable software. It is often described as an extension of C, but over time it has grown into a language of its own, supporting multiple programming styles including procedural, object-oriented, and even generic programming.

At its core, C++ is about **control and efficiency**. When you write a C++ program, you are not just telling the computer *what* to do, but also influencing *how* it does it. You can manage memory directly, control performance, and decide how data is stored and accessed. This makes C++ both powerful and demanding. It does not hide complexity from you, but instead gives you the tools to understand and handle it.

Because of this, learning C++ is like learning how the engine of a machine works rather than just driving it. Once you understand it, other languages start to feel more intuitive.



History Of C++:

Like other programming languages popular today, C++ has quite a story behind it, stretching back to 1979. **Bjarne Stroustrup**, a Danish computer scientist working at Bell Labs, started work on a new programming language called **'C with Classes'** deriving from the original C language created by Dennis Ritchie

A large inspiration for the language was Simula, which is considered to be the first object-oriented programming language ever. While Stroustrup was programming for his Ph.D. thesis, he found Simula pleasant to program in, but the language was too slow for practical use. Therefore, he set out to add Simula-like features to C, thus making a language that was both performant and relatively high-level for the time. In this way, the 'C with Classes' programming language was born. Initially, C with Classes just added features to the C compiler.

In 1982, Stroustrup started to build the next version of his language. He built a new compiler based on the existing C compiler and added a ton of new features. After cycling through a couple of names, he ended up with 'C++'. The name comes from the C's incrementation operator, it means C plus one. In 1985, the first commercial implementation of C++ was released. The same year also saw the release of the first reference book of the language - The C++ Programming Language written by the creator himself

Stroustrup wanted the addition of object-oriented programming (OOP), which drove him to create C++. Therefore, the inclusion of OOP is the significant difference between C and C++. To summarize his interview with Big Think, the motivation for making C++ was increasing a program's maintainability.

To Conclude:

Although C++ is used in many industries and can be used to write almost anything, it particularly excels in delivering performance and using system resources efficiently. As we have said, it's an extension of the C programming language, therefore it retains a strong link to C, and will compile most C programs. Isn't that WONDERFUL?

Oh! That was quite a theory and a flashback into C++'s history. But, before you learn anything, you should know how it came into existence. From here on, we will move ahead to the more technical side and slowly start writing some code.

Difference Between C and C++:

To understand C++, it helps to see where it comes from. C is a procedural programming language, meaning it focuses on functions and step-by-step instructions. Programs in C are typically organized around functions that operate on data.

C++, on the other hand, expands this idea by introducing **object-oriented programming (OOP)**. Instead of only thinking in terms of functions, you can organize programs around **objects**, which combine data and behaviour together. This makes it easier to model real-world systems and manage large codebases.

Another key difference is that C++ provides additional features such as classes, inheritance, polymorphism, and function overloading. These features allow developers to write more modular and reusable code. While C gives you the raw tools, C++ adds layers that help you structure complex programs more effectively.

Despite these differences, C++ still retains most of C's capabilities. This means you can write low-level, efficient code just like in C, while also taking advantage of higher-level abstractions when needed. In a way, C++ is like a toolkit that includes both basic tools and advanced machinery, and it is up to the programmer to decide what to use.

Features of C++:

A lot of developers in different communities argue about the best programming language. Actually, it is impossible to identify which tool or technology is indeed the optimum variant for ALL problems since different languages are designed for different kinds of problems.

Nevertheless, the C++ language has some powerful arguments up its sleeve. In this section, we will look at some of the features of C++ and try to understand what makes it so popular and a top choice for various projects.

- **Performance**: Performance is directly related to speed. When it comes to speed, C++ has no match in today's list of popular languages. C++ is the Flash of the world of programming!

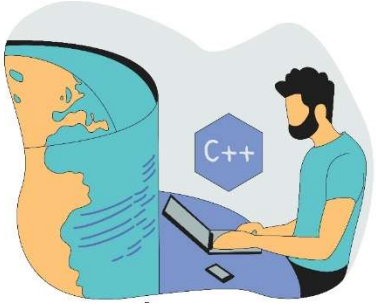
  It is one of the fastest and most predictable languages out there, only contested by other low-level programming languages like Rust. The control provided by C++ over the system resources enables a skilled coder to write a program that is quicker and more powerful than a similar program written in another programming language.

  Interesting Fact: Most AAA game titles are written using C++ because these titles push the limits of existing hardware, so resource efficiency is very important.

- **Flexibility**: C++ is a multi-paradigm programming language. This means that it supports other styles such as procedural programming, in addition to object-oriented programming.

  These paradigms are essentially different ways of looking at and solving a problem. Two different C++ programmers can solve the same problem in different ways. Paradigms can also be combined to get the most efficient result.

  On the other side, having different ways of solving problems makes C++ more complex, but at the same time, more powerful as well.

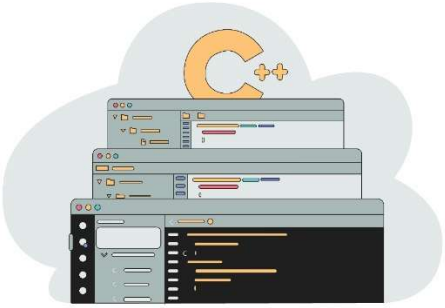


- **Syntax**: When it comes to syntax, it is a bit harder to choose the best option. Today, the majority of popular programming languages have pretty nice syntax. Unlike some other languages, C++ has quite an intuitive syntax, which is actually not so hard to comprehend, by looking at the code a bit more creatively.

  Hence, the prize named "Easiest Syntax" doesn't go to C++, since it has more elements to write in contrast to some other languages. Fortunately, even with more code and intuitive syntax, C++ code is more structured and follows the same set of rules. Therefore, despite its length, C++ has a pretty straightforward syntax.

- **Large Ecosystem**: The language has been around for almost 40 years, which means that most of the software problems have already been solved by open-source libraries and frameworks. C++ has a ton of developers that use it, upgrade it, and write open-source libraries.

  Therefore, in simple words, while learning or using the language, you can draw on the work done by these people already.



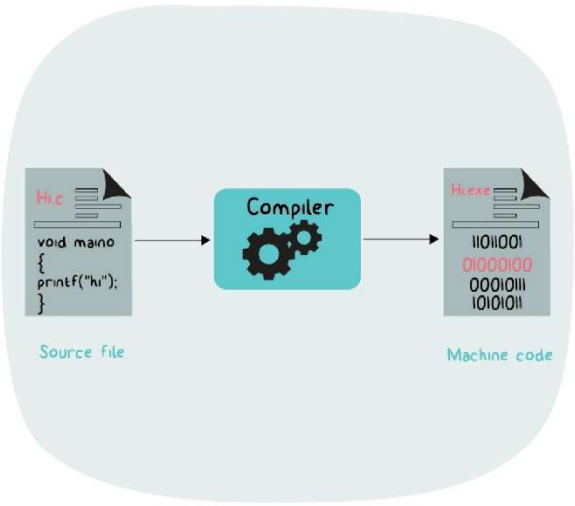
C++ is an exceptional language when it comes to its abilities. It lets us do virtually anything possible and impossible. It is a technology that can be used to accomplish a tremendous variety of tasks.

A large number of jobs available is a reflection of the long-standing popularity of C++. Not only do many new applications get coded in C++, but the huge range of programs already built during the language necessitate hiring coders to keep them current and updated. Hence, C++ developers are welcome to nearly every software development company.

How C++ Works?

When you write a C++ program, you do not directly communicate with the computer's hardware. Instead, you write code in a human-readable form called **source code**, usually saved in files with a .cpp extension. This code must be translated into a form that the computer can understand and execute.

This translation happens through a process called **compilation**.



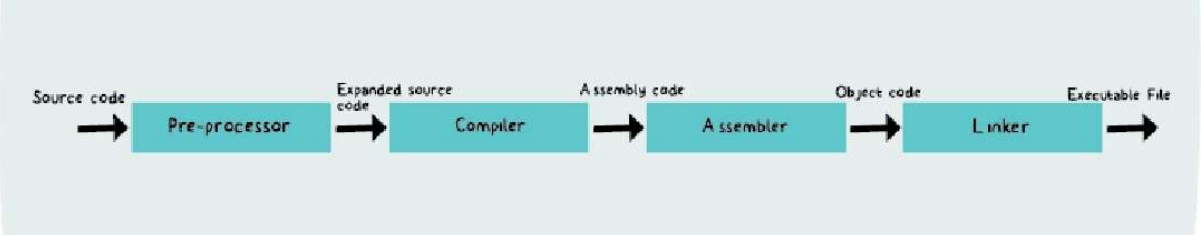
Compiled languages are converted directly into machine code that the processor can execute. As a result, they tend to be faster and more efficient to execute than interpreted languages.

Steps in Compilation:

The first step involves the compiler taking your source code and checking it for errors. If the code is valid, it is converted into an intermediate form called **object code**. This object code is not yet a complete program, but it contains machine-level instructions.

In the next step, a **linker** combines this object code with other necessary pieces, such as libraries, to produce the final **executable file**. This executable file is what you actually run on your system.

So, the journey of a program can be thought of as a transformation:



Where is C++ used?

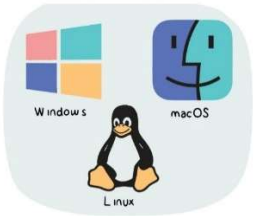
Remember, C++ is a general-purpose programming language? This means it is used in most places! And that's the reason why C++ has held its grip even after 35+ years of its release. C++ can be used for; Operating Systems Development, Game Development, Computer Graphics, Cloud/Distributed Systems, Compiler Development, And more... The list goes on! Let's dive deep.

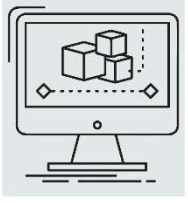
- **Operating Systems:** Be it Microsoft Windows or Mac OSX or Linux - all of them are programmed in C++. No doubt, right?

  Speed and security are the main factors for an OS, and C++ is the master of those features, C/C++ is the backbone of all well-known operating systems owing to the fact that it is a strongly typed and fast programming language which makes it an ideal choice for developing an operating system.
- **Game Development**: Game Development is one of the fields that is said to have one of the brightest futures in the upcoming decade. Well, believe it or not, C++ is a direct ticket to a billion-dollar game development industry.

  To tackle big games in the larger gaming companies, knowing C++ is critical. It's fast, the compilers and optimizers are solid, and you get a lot of control over memory management. It has extensive libraries, which come in handy for designing and powering complex graphics.
- **Computer Graphics**: All graphics applications require fast rendering and just like the case of web browsers, C++ helps in reducing the latency. Software that employs computer vision, digital image processing, high-end graphical processing - they all use C++ as the backend programming language.

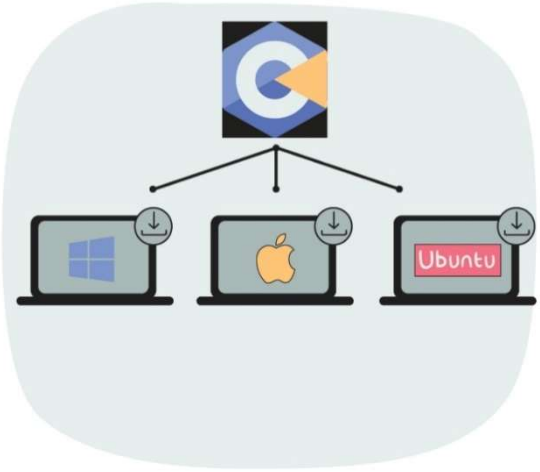
  Even the popular games that are heavy on graphics use C++ as the primary programming language. The speed that C++ offers in such situations helps the developers in expanding the target audience because an optimized application can run even on low-end devices that do not have high computation power available.
- **Cloud/Distributed Systems**: Large organizations that develop cloud storage systems and other distributed systems also use C++ because it connects very well with the hardware and is compatible with a lot of machines. Cloud storage systems use scalable file systems that work close to the hardware.
- **Compiler Development**: A compiler is responsible for not only converting the source code to machine code but also performs various checks to ensure the code is correct and error-free. The compilers of various programming languages use C and C++ as the main programming language.




Installing C++ and its IDE:

To work with C++ on your machine, first, you will need to install the C++ Compiler in your system. It is unlikely that your system will be shipped with C++ Compiler.



Almost every C++ programmer uses the simplest way for getting this by using the GCC compiler. But first, would you like to know what GCC is? GCC stands for GNU Compiler Collection, it is an open-source compiler that supports the execution of multiple programming languages like C, C++, Fortran, Golang, and more.

GCC has been ported to multiple platforms and hence it helps in building C++ programs on a wide variety of systems. Currently, GCC's appropriate version for multiple platforms is available widely and for free.

But we are going to use something more efficient i.e., “Codeblocks”.

Installing Code::Blocks with MinGW:

Before we start writing C++ programs, we need a place to write and run them. For this, we will use Code::Blocks, which comes with a built-in C/C++ compiler called MinGW.
Follow the steps carefully.

Step 1: Download Code::Blocks

- Open your browser
- Search for: Code::Blocks official website
- Open the website:
  👉 Code::Blocks
- Go to the Downloads section
- Click on “Download the binary release”

Step 2: Choose the Correct Version

You will see multiple options. Choose the one that includes MinGW.

- Look for a file named similar to:

  codeblocks-20.03mingw-setup.exe

⚠️ Important:

- Always choose the version with "mingw" in the name
- This version already contains the C compiler
- No need to install anything separately

Step 3: Install Code::Blocks

- Double-click the downloaded .exe file
- Click Next → I Agree → Next
- Keep default settings
- Click Install

  Once installation completes:
- Click Finish

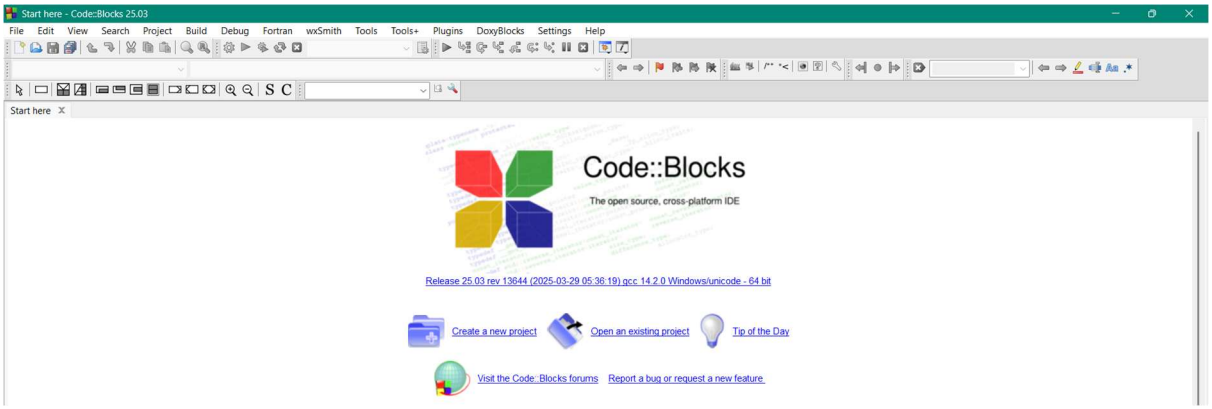
Step 4: First Launch Setup

- Open Code::Blocks
- It may ask to select a compiler

If prompted:

- Choose: GNU GCC Compiler
- Click Set as default
- Click OK

The setup is done and screen will look like this:



Creating files in Code::Blocks:

Step 1: Create Your First Project

- Click on Create a new project
- Select Console Application
- Click Go → Next
- Choose language: C++
- Click Next

Step 2: Set Project Details

- Enter a Project Title (e.g., MyFirstProgram)
- Choose a folder location
- Click Next → Finish

Step 3: Write Your First Program
You will see a default file (main.cpp).

- Inside it, you can write your first program
- Or modify the existing code

Step 4: Build and Run the Program

- Click on Build and Run (green play button)
  OR press F9
- Output will appear in a console window

At this stage, do not worry about:

- How the compiler works internally
- Complex settings
- Advanced configurations

Right now, your goal is simple:

Write → Run → Observe

First C++ Program & Boilerplate

Let's begin with the classic first step into any programming language. It may look small, but it opens the door to everything that follows.

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!";
    return 0;
}
```

When this program runs, it simply prints:

Hello, World!

At first glance, this might feel like a few strange symbols stitched together. But each line has a purpose, and together they form the basic skeleton of almost every C++ program you will write.

Understanding the Structure of a C++ Program:

A C++ program is not just random lines of code. It follows a structure, almost like a recipe. Each part has a role, and removing or misplacing one piece can break the whole program.

Let's gently unwrap each component.

1. #include <iostream>: Bringing in Tools

This line is a **preprocessor directive**. It tells the compiler to include a file before the program is compiled. The file iostream stands for **input-output stream**. It contains definitions for objects that allow your program to take input and display output.

Think of it like opening a toolbox before starting work. Without it, your program would not know how to print anything on the screen.

2. What is iostream?

iostream is a standard library file in C++ that provides functionality for input and output operations.

It gives you access to:

- cout → for output (printing to screen)
- cin → for input (taking data from user)

Without including iostream, using cout or cin would result in an error because the compiler would not recognize them.

3. What is std?

In C++, many standard features are grouped under something called a **namespace**. The standard library belongs to the namespace called std (short for *standard*).

So, when you write: std::cout you are telling the compiler: "Use cout from the standard namespace."

This avoids confusion in large programs where different libraries might have functions or objects with the same name.

4. int main(): The Starting Point

Every C++ program begins execution from the main() function.

```
int main() {
   // code
}
```

- main is the entry point of the program
- int means the function returns an integer value
- The {} braces contain the body of the program

No matter how big your program becomes, execution always starts from here.

5. cout: Displaying Output

Inside the main() function, we wrote: std::cout << "Hello, World!";

Here:

- cout stands for **console output**
- << is the insertion operator (it sends data to output)
- "Hello, World!" is the text to display

So this line simply tells the computer: "Print this message on the screen."

6. return 0; Ending the Program

At the end of main(), we often write: return 0;

This indicates that the program has executed successfully.

- 0 typically means success
- Any non-zero value can indicate an error

While modern compilers may allow you to omit it, it is good practice to include it for clarity.

Making It Simpler: Using namespace std

Writing std::cout every time can feel repetitive, especially for beginners. C++ allows you to simplify this using: using namespace std;

This tells the compiler: "Use the standard namespace by default."
Now you can write a cleaner version of the same program.

C++ Boilerplate (Basic Template):

```
1  #include <iostream>
2
3  int main() {
4      // Program codes goes here
5      return 0;
6  }
```

This is the **basic boilerplate** of a C++ program.

Final Thought:

If a C++ program were a stage play:

- #include opens the backstage doors
- iostream brings in the actors for input/output
- std tells you which troupe they belong to
- main() is where the play begins
- cout is the actor speaking to the audience

Right now, it's just one line of dialogue. Soon, you'll be directing entire performances.

Escape Sequences & Comments

// Comments in C++:

When you write programs, you are not just communicating with the computer, but also with yourself and others who may read your code later. This is where **comments** come in. Comments are lines in a program that are ignored by the compiler. They do not affect how the program runs, but they make the code easier to understand.

In real-world programming, code is rarely written once and forgotten. It is read, modified, debugged, and improved over time. Comments act like small notes left in the code, explaining *why something is done*, not just *what is written*. A well-commented program feels guided, while a program without comments can feel like walking through a maze without signs.

C++ provides two types of comments:

- **Single-line comments**, which start with // and continue till the end of the line
- **Multi-line comments**, which start with /* and end with */, allowing you to write comments across multiple lines

Comments are especially useful for:

- Explaining logic
- Temporarily disabling code during testing
- Making programs readable for others

Lesson Program: Comments in C++

Source Code: Comments.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Printing a welcome message
    cout << "Welcome to C++ Programming!\n";

    // Printing another line
    cout << "This program demonstrates comments usage.\n";

    /*
        Multi-line comment:
        The following lines perform simple addition
        and display the result
    */

    int a = 10;   // First number
    int b = 20;   // Second number
    int sum = a + b;   // Adding numbers

    // Displaying the result
    cout << "Sum = " << sum << endl;

    return 0;   // Program ends successfully
}
```

// Escape Sequence:

When printing text, sometimes we need more control than just plain words. We may want to move to a new line, insert a tab space, print quotation marks, or format the output neatly. This is where **escape sequences** come into play.

An escape sequence is a combination of characters that begins with a backslash \. It tells the compiler to perform a special action instead of printing the characters as they are. Think of escape sequences as hidden instructions embedded inside strings. They allow you to control how the output appears on the screen.

Why Do We Need Escape Sequences?

If you write: `cout << "Hello World";` everything appears on one line.
But what if you want:

```
Hello
World
```

You cannot just press Enter inside the string. Instead, you use:

```
cout << "Hello\nWorld";
```

Here, `\n` tells the program to move to a new line.

Where and How Are They Used?

Escape sequences are mainly used inside **string literals** (text inside quotes " "). They modify how the output is displayed.

For example:

- Formatting output
- Printing special characters
- Structuring readable console output

List of Escape Sequences in C

Here are the most useful escape sequences:

- `\n` → New line
- `\t` → Horizontal tab (space gap)
- `\b` → Backspace (removes previous character)
- `\r` → Carriage return (moves cursor to beginning of line)
- `\\` → Prints a backslash \
- `\"` → Prints double quotes "
- `\'` → Prints single quote '
- `\0` → Null character (end of string)

Lesson Program: Escape Sequence in C++
Source Code: EscapeSequence.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Demonstrating Escape Sequences in C++

    cout << "Demonstrating Escape Sequences in C++\n\n";

    // 1. New Line
    cout << "1. New Line (\\n):\n";
    cout << "Hello\nWorld\n\n";

    // 2. Tab Space
    cout << "2. Tab Space (\\t):\n";
    cout << "Hello\tWorld\n\n";

    // 3. Backspace
    cout << "3. Backspace (\\b):\n";
    cout << "Helloo\b World\n\n";

    // 4. Carriage Return
    cout << "4. Carriage Return (\\r):\n";
    cout << "Hello World\rHi\n\n";

    // 5. Backslash
    cout << "5. Backslash (\\\\):\n";
    cout << "\\\n\n";

    // 6. Single Quote
    cout << "6. Single Quote (\\'):\n";
    cout << "\'\n\n";

    // 7. Double Quote
    cout << "7. Double Quote (\\\"):\n";
    cout << "\"C++\"\n\n";

    // 8. Alert (may produce sound)
    cout << "8. Alert (\\a):\n";
    cout << "\a\n";

    return 0;    // Program ends
}
```

The Language Building Blocks: Tokens

Before a C++ program can be understood by the compiler, it must be broken down into smaller pieces. These smallest individual units of a program are called **tokens**. Just like a sentence is made up of words and punctuation, a C++ program is made up of tokens. The compiler reads your code not as full lines or paragraphs, but as a sequence of these meaningful building blocks.

Whenever you write a program, whether it is a simple cout statement or a complex function, everything you type is ultimately divided into tokens. This is why understanding tokens helps you understand how the compiler "sees" your code.

In simple terms, **a token is the smallest unit in a C++ program that has meaning to the compiler**.

Why Are Tokens Important?

Tokens are important because they form the basic structure of any program. If tokens are used incorrectly, the compiler will not understand the code and will generate errors. For example, writing a variable name incorrectly or misusing an operator can break the entire program.

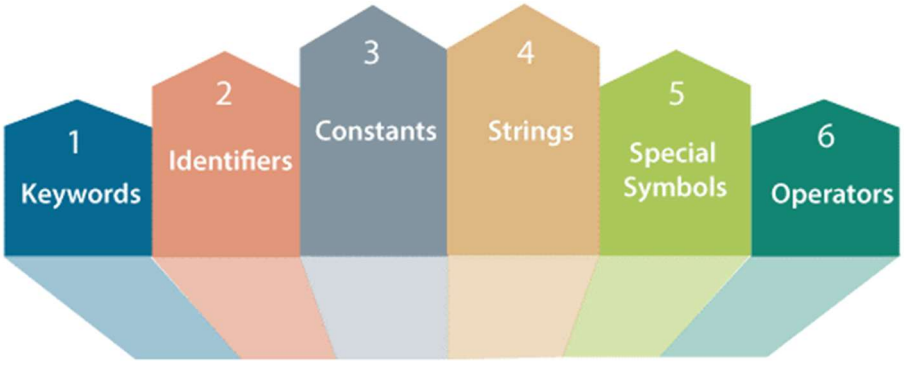
By learning tokens, you begin to see programming not as random syntax, but as a well-organized system where every symbol has a purpose. It also helps in debugging, because you can identify exactly which part of the code is causing an issue.

Types of Tokens in C++:

C++ mainly has **six types of tokens**, each playing a specific role:

- **Keywords**: Reserved words with special meaning (e.g., int, return, if)
- **Identifiers:** Names given to variables, functions, etc. (e.g., sum, main)
- **Constants (Literals):** Fixed values (e.g., 10, 3.14, 'A')
- **Strings**: Sequence of characters inside quotes (e.g., "Hello")
- **Operators**: Symbols that perform operations (e.g., +, -, *, =)
- **Punctuators (Special Symbols)**: Symbols used for structure (e.g., ;, {}, ())

Together, these tokens form every valid C++ program.



Character Set of C++

To form tokens, C++ uses a defined set of characters known as the **character set**. These are the basic symbols that you can use to write any program.

The C++ character set includes:

- Uppercase: A to Z
- Lowercase: a to z
- Digits: 0 to 9
- Special characters: ! " # $ % & ' ( ) * + - . : , ; ` ~ = < > { } [ ] ^ _ \ /
- Other special characters: Blank space, tab, etc. (Compiler ignores white spaces unless they are part of string)

These characters combine to form tokens, and tokens combine to form complete programs.

// Keywords & Identifiers:

In every C++ program, the words you write are not all treated the same. Some words are already defined by the language itself, while others are created by the programmer. This brings us to two important types of tokens: **keywords** and **identifiers**.

C++ Keywords:

Keywords are **reserved words** that have a fixed meaning in C++. They are predefined by the language and cannot be used for any other purpose, such as naming variables or functions.

For example, words like int, float, return, and if are keywords. Each of these has a specific role. When you write int, the compiler immediately understands that you are declaring an integer variable. These meanings are built into the language and cannot be changed.

C++ has a larger set of keywords compared to C because it supports advanced features like object-oriented programming. All keywords are written in lowercase and must be used exactly as defined.

Keywords act like the grammar of the language. They give structure to your program and tell the compiler how to interpret different parts of the code.

List of Keywords:

asm	auto	bool	break	case	catch
char	class	const	const_cast	condition	default
delete	do	double	dynamic_cast	else	enum
explicit	export	extern	false	float	for
friend	goto	if	inline	int	long
mutable	namespace	new	operator	private	protected
public	register	reinterpret_cast	return	short	signed
sizeof	static	static_cast	struct	switch	template
this	throw	true	try	typedef	typeid
typename	union	unsigned	using	virtual	void
volatile	wchar_t	while			

C++ Identifiers:

Identifiers are the names given to elements created by the programmer. These include variables, functions, arrays, classes, and more.

For example: int age; float height;

Here, age and height are identifiers. They are chosen by the programmer to represent data in a meaningful way. Unlike keywords, identifiers do not have predefined meanings. Their purpose depends entirely on how they are used in the program. Good identifiers make code more readable and easier to understand.

Rules for Defining Identifiers in C++:
While identifiers give you flexibility, there are certain rules that must be followed:

1. An identifier must begin with a letter (A–Z or a–z) or an underscore (_)
2. It can contain letters, digits (0–9), and underscores
3. It cannot start with a digit
4. It cannot be a keyword
5. Identifiers are case-sensitive (age and Age are different)
6. No special symbols like @, #, %, or spaces are allowed

✔ Example Valid Identifiers:

- sum, totalMarks, _count, Age1

❌ Example Invalid Identifiers:

- int (keyword), 2ndValue (starts with digit), my-name (special character)

Lesson Program: Keywords and Identifiers in C++
Source Code: KeywordsAndIdentifiers.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // int, float, char are keywords
    // age, height, grade are identifiers

    int age = 23;          // 'int' is a keyword, 'age' is an identifier
    float height = 5.9;    // 'float' is a keyword, 'height' is an identifier
    char grade = 'A';      // 'char' is a keyword, 'grade' is an identifier

    // cout and endl are part of standard library (used for output)

    cout << "Age: " << age << endl;
    cout << "Height: " << height << endl;
    cout << "Grade: " << grade << endl;

    // return is a keyword
    return 0;
}
```

// Constants and Variables in C++

Every C++ program works with **data**. Data is the information that a program stores, processes, and displays. In C++, this data is mainly classified into two types: **constants** and **variables**.

Constants in C++:

A constant is a **fixed value** that does not change during the execution of a program. Once defined, its value remains the same throughout. Constants are used when certain values are meant to stay unchanged, such as mathematical values or fixed quantities.

Constants can be of different types:

- **Integer constants** → 10, -25, 0
- **Floating-point constants** → 3.14, -0.001, 2.5e3
- **Character constants** → 'A', 'z', '1'
- **String constants** → "Hello", "C++ Programming"

In C++, constants can also be defined using the const keyword.
Example:

```
const int days = 7;
```

Here, days is a constant whose value cannot be changed.

Variables in C++:

A variable is a **named location in memory** used to store data that can change during program execution. Unlike constants, variables can be modified as the program runs. Before using a variable, it must be declared with a data type, which tells the compiler what kind of data it will store.

Example:

```
int age = 20;
float height = 5.9;
char grade = 'A';
```

Here:

- age, height, and grade are variables
- Their values can change during execution

Variables are essential because they allow programs to store and manipulate dynamic data.

Rules for Variables

The rules for naming variables in C++ are the same as identifiers:

- Must begin with a letter or underscore
- Can contain letters, digits, and underscores
- Cannot be a keyword
- Case-sensitive
- No special symbols or spaces allowed

Lesson Program: Constants and Variables in C++
Source Code: ConstantsAndVariables.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Constant (value cannot be changed)
    const float PI = 3.14;

    // Variables (values can change)
    int days = 7;
    int age = 23;
    float height = 5.9;
    char grade = 'A';

    // Displaying constant value
    cout << "PI = " << PI << endl;

    // Displaying variable values
    cout << "Days in a week = " << days << endl;
    cout << "Age = " << age << endl;
    cout << "Height = " << height << endl;
    cout << "Grade = " << grade << endl;

    return 0;
}
```

// Constants, Strings, Operators, and Punctuators in C++:

In addition to keywords and identifiers, C++ programs are built using other important tokens such as **constants, strings, and punctuators**. These tokens help represent values, text, and the structure of a program. Each of them plays a simple but essential role in writing meaningful code.

Constants in C++:

Constants are **fixed values** that do not change during the execution of a program. Once a constant is defined, its value remains the same throughout.

For example: 10, 3.14, 'A'

These values are directly used in programs without needing a variable. Constants can be of different types:

- **Integer constants** → 10, 100
- **Floating-point constants** → 3.14, 2.5
- **Character constants** → 'A', 'b'

Constants make programs simple when values do not need to change. They are used to represent fixed data such as marks, grades, or predefined numbers.

Strings in C++:

Strings are a sequence of characters enclosed in **double quotes**.

For example: "Hello", "C++ Programming"

Strings are mainly used to display messages or text in a program. They are commonly used with output statements like cout. Unlike character constants (which use single quotes), strings always use double quotes and can contain multiple characters.

Operators in C++:

Operators are **symbols used to perform operations** on data. They tell the compiler to carry out specific tasks such as addition, comparison, or assignment.

For example: +, -, *, /, =

Operators are used to:

- Perform calculations
- Compare values
- Assign values

They are essential for writing logic in programs.

Punctuators in C++:

Punctuators are **special symbols** used to organize and structure a C++ program. They help the compiler understand how different parts of the code are separated and grouped.

Some common punctuators include: ; ( ) { } , [ ] .

For example:

- ; marks the end of a statement
- { } define a block of code
- ( ) are used in functions

Without punctuators, the structure of a program would be unclear, and the compiler would not be able to interpret it correctly.

Lesson Program: Other tokens in C++
Source Code: OtherTokens.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Constants
    int a = 10;          // 10 is an integer constant
    int b = 5;           // 5 is an integer constant
    char ch = 'A';       // 'A' is a character constant

    // String
    cout << "Welcome to C++ Programming\n";

    // Operators
    int sum = a + b;     // '+' is an operator
    int diff = a - b;    // '-' is an operator

    // Displaying values
    cout << "Sum = " << sum << endl;
    cout << "Difference = " << diff << endl;
    cout << "Character = " << ch << endl;

    // Punctuators used:
    // ;  -> end of statement
    // {} -> block of code
    // () -> function

    return 0;
}
```

Variables and Data Types in C++

Until now, programs were mainly printing messages. But real programs do much more than that. They **store values, use them, and sometimes change them over time**.

Think about simple real-life situations:

- A student has marks
- A shop has prices
- A program calculates a result

In all these cases, some values need to be stored and used later. A computer does not "remember" things like we do. Instead, it stores everything in its memory. This is where **variables** come into the picture.

What is a Variable?

A variable is a **named location in memory** where a value is stored. When a program runs, the computer uses its memory to store data. Instead of remembering memory locations (which are complex), we give them simple names. These names are called **variables**.

In simple terms: A variable is a **container that stores data**

Understanding Variables with Memory:

You can imagine memory like a collection of boxes 📦 :

- Each box has a **name** → variable
- Each box stores a **value**
- Each box has a **type** → defines what kind of data it can hold

For example: `int age = 21;`

Here:

- int → type (integer)
- age → variable name
- 21 → value stored

Declaring a Variable:

Declaration means telling the compiler: "I want to create a variable of this type with this name"

```
int age;
float salary;
char grade;
```

At this stage:

- Memory is reserved
- No value is stored yet

Assigning a Value:

Assignment means giving a value to a variable.

```
age = 21;
salary = 55000.0;
grade = 'A';
```

This stores the values in memory.

Initializing a Variable:

Initialization means declaring and assigning a value at the same time.

```
int year = 2025;
float pi = 3.14;
char section = 'B';
```

This is the most common and recommended way.

Where Are Variables Used?

Variables are used:

- To store input values
- To perform calculations
- To hold results
- To control program flow

Almost every program depends on variables.

Rules for Naming Variables:

Variable names follow the same rules as identifiers:

- Must start with a letter (a–z, A–Z) or underscore (_)
- Can contain letters, digits, and underscores
- Cannot start with a digit
- Cannot be a keyword
- No spaces or special symbols
- Case-sensitive (age and Age are different)

Making a Variable Constant

Sometimes, we do not want a value to change. In such cases, we use **constants**. C++ provides **two ways** to create constants:

1. Using const Keyword:

```
const float PI = 3.14;
```

- Value cannot be changed later
- Type-safe (follows data type rules)

2. Using #define (Preprocessor Constant):

```
#define DAYS 7
```

- Defined before compilation
- No data type is specified
- Replaces value everywhere in code

Difference (Basic Idea):

- const → behaves like a variable (recommended in C++)
- #define → simple text replacement

Lesson Program: Variables in C++

Source Code: Variables.cpp

```cpp
#include <iostream>
using namespace std;

// Preprocessor constant
#define DAYS 7

int main() {

    // Declaration
    int age;
    float salary;
    char grade;

    // Assignment
    age = 21;
    salary = 55000.0;
    grade = 'A';

    // Initialization
    int year = 2025;
    float pi = 3.14;
    char section = 'B';

    // Constant using const
    const float PI = 3.14159;

    // Using variables
    cout << "Age: " << age << endl;
    cout << "Salary: " << salary << endl;
    cout << "Grade: " << grade << endl;

    cout << "Year: " << year << endl;
    cout << "Pi: " << pi << endl;
    cout << "Section: " << section << endl;

    // Using constants
    cout << "Days in a week (#define): " << DAYS << endl;
    cout << "Value of PI (const): " << PI << endl;

    return 0;
}
```

// Data Types in C++

In C++, every variable stores some kind of data. But not all data is the same. Some values are whole numbers, some are decimal numbers, and some are characters or text. To handle this properly, C++ uses **data types**.

A **data type** defines:

- What kind of data a variable can store
- How much memory it will use
- What range of values it can hold

In simple terms: A data type tells the computer **what type of value is being stored**

Why Do We Need Data Types?

Computers manage memory very carefully. If you store a small number, it should not take too much space. If you store a large number or decimal value, more memory may be needed.

Data types help:

- Use memory efficiently
- Perform correct operations
- Avoid errors

For example, storing 10 is different from storing 10.5, and both are different from storing 'A'.

Classification of Data Types in C++

1. Primary (Fundamental) Data Types in C++:

Primary data types are the basic built-in types supported directly by the compiler.

They include:

- int → stores integers (e.g., 10, -5)
- float → stores decimal values (e.g., 3.14)
- double → stores decimal values with higher precision
- char → stores a single character (e.g., 'A')
- bool → stores logical values (true or false)

2. Derived Data Types:

Derived data types are formed using primary data types.

Examples:

- Arrays
- Pointers
- Functions

They help in handling collections of data and memory.

3. User-defined Data Types:

User-defined data types are created by the programmer.

Examples:

- struct
- union
- enum
- typedef

They help in organizing complex data.

Note on Size and Range

Each data type uses a specific amount of memory and has a defined range. This may vary depending on the system and compiler, but the purpose remains the same: Efficient storage and proper data handling

Lesson Program: Data Types in C++

Source Code: DataTypes.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Primary data types

    int age = 23;              // Integer type
    float pi = 3.14;           // Float type
    double big = 123456.789;   // Double type
    char grade = 'A';          // Character type
    bool isStudent = true;     // Boolean type

    // Displaying values
    cout << "Integer (age): " << age << endl;
    cout << "Float (pi): " << pi << endl;
    cout << "Double (big): " << big << endl;
    cout << "Character (grade): " << grade << endl;
    cout << "Boolean (isStudent): " << isStudent << endl;

    return 0;
}
```